Assignment_6_709

Saurabh Mishra

2026-03-19

Problem set 6: Model Selection

To demonstrate the application of subset selection methods

Consider the “Hitters” data. For loading the data you need to install ISLR library. We wish to predict a baseball player’s salary on the basis of various predictors. So first we need to choose an optimal model.

```
library(ISLR)

## Warning: package 'ISLR' was built under R version 4.5.2

library(leaps)

## Warning: package 'leaps' was built under R version 4.5.2

set.seed(123)

head(Hitters)

##                    AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits
CHmRun
## -Andy Allanson       293   66     1   30  29    14     1    293    66
1
## -Alan Ashby          315   81     7   24  38    39    14   3449   835
69
## -Alvin Davis         479  130    18   66  72    76     3   1624   457
63
## -Andre Dawson        496  141    20   65  78    37    11   5628  1575
225
## -Andres Galarraga    321   87    10   39  42    30     2    396   101
12
## -Alfredo Griffin     594  169     4   74  51    35    11   4408  1133
19
##                    CRuns CRBI CWalks League Division PutOuts Assists Errors
## -Andy Allanson        30   29     14      A        E     446      33     20
## -Alan Ashby          321  414    375      N        W     632      43     10
## -Alvin Davis         224  266    263      A        W     880      82     14
## -Andre Dawson        828  838    354      N        E     200      11      3
## -Andres Galarraga     48   46     33      N        E     805      40      4
## -Alfredo Griffin     501  336    194      A        W     282     421     25
##                    Salary NewLeague
## -Andy Allanson         NA         A
## -Alan Ashby         475.0         N
```

```
## -Alvin Davis       480.0         A
## -Andre Dawson      500.0         N
## -Andres Galarraga   91.5         N
## -Alfredo Griffin   750.0         A
```

1. Clean the data by removing all rows that have missing value in any variable. You will be working with this clean data.

```
hitter_clean = na.omit(Hitters)
head(hitter_clean)
```

```
##                    AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits
CHmRun
## -Alan Ashby          315   81     7   24  38    39    14   3449   835
69
## -Alvin Davis         479  130    18   66  72    76     3   1624   457
63
## -Andre Dawson        496  141    20   65  78    37    11   5628  1575
225
## -Andres Galarraga    321   87    10   39  42    30     2    396   101
12
## -Alfredo Griffin     594  169     4   74  51    35    11   4408  1133
19
## -Al Newman           185   37     1   23   8    21     2    214    42
1
##                    CRuns CRBI CWalks League Division PutOuts Assists Errors
## -Alan Ashby          321  414    375      N        W     632      43     10
## -Alvin Davis         224  266    263      A        W     880      82     14
## -Andre Dawson        828  838    354      N        E     200      11      3
## -Andres Galarraga     48   46     33      N        E     805      40      4
## -Alfredo Griffin     501  336    194      A        W     282     421     25
## -Al Newman            30    9     24      N        E      76     127      7
##                    Salary NewLeague
## -Alan Ashby         475.0         N
## -Alvin Davis        480.0         A
## -Andre Dawson       500.0         N
## -Andres Galarraga    91.5         N
## -Alfredo Griffin    750.0         A
## -Al Newman           70.0         A
```

2. Split the data randomly into train-test keeping 80% as training sample and remaining as test sample.

```
train = sample(1:nrow(hitter_clean), 0.8*nrow(hitter_clean))
train
```

```
##   [1] 179  14 195 118 229 244 262 260 153  90  91 197 243 253 185  92 137
221
##  [19]  99  72  26   7 170 247 211 164  78  81  43 103 117  76 143  32 109
242
##  [37] 240 169  74  23 155 188  53 135 246 223 166  34  69 258 232  63 141
```

```
210
##  [55]  97 250 255  38  21  41 175 254  60  16 116  94   6  86 196  39 159
256
##  [73]  50 216   4  13 215 127 207  52  22  89 160  25  35 168 112  30 140
193
##  [91] 121 110 158  64 142  67 151 122  79  85 136  51 225 106  98 161 186
17
## [109]  46  54 172 198 165  24 113 107 220 102 147   5  70 200 150 230 205
55
## [127]  75  83 194 154  48  77 163 111 145  68   1  40 176 152 199 182 226
88
## [145] 134 227 133  20 104 178  93  36 184 108 183  49  42  59  84  11 138
8
## [163] 218 238  66 132 101 231 214  44 203 177  45 257 173  87  33 126 261
10
## [181]  95  82   9 228  80  58  61 171 239 212 119 224 251 120  57  29 105
219
## [199]  73 222 235 115 180 130  71 167 174 131 181 149
```

```
train_data = hitter_clean[train, ]
test_data = hitter_clean[-train, ]
```

```
head(train_data)
```

```
##                AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
CRuns
## -Marvell Wynne   288   76     7   34  37    15     4   1644   408     16
198
## -Bill Almon      196   43     7   29  27    30    13   3231   825     36
376
## -Rick Cerone     216   56     4   22  18    15    12   2796   665     43
266
## -Jack Howell     151   41     4   26  21    19     2    288    68      9
45
## -Spike Owen      528  122     1   67  45    51     4   1716   403     12
211
## -Tom Paciorek    213   61     4   17  22     3    17   4061  1145     83
488
##                CRBI CWalks League Division PutOuts Assists Errors Salary
## -Marvell Wynne  120    113      N        W     203       3      3    240
## -Bill Almon     290    238      N        E      80      45      8    240
## -Rick Cerone    304    198      A        E     391      44      4    250
## -Jack Howell     39     35      A        W      28      56      2     95
## -Spike Owen     146    155      A        W     209     372     17    350
## -Tom Paciorek   491    244      A        W     178      45      4    235
##                NewLeague
## -Marvell Wynne         N
## -Bill Almon            N
## -Rick Cerone           A
## -Jack Howell           A
```

```
## -Spike Owen            A
## -Tom Paciorek          A
```

```r
head(test_data)
```

```
##                 AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun
CRuns
## -Alvin Davis     479  130    18   66  72    76     3   1624   457     63
224
## -Andre Dawson    496  141    20   65  78    37    11   5628  1575    225
828
## -Andy VanSlyke   418  113    13   48  61    47     4   1512   392     41
205
## -Buddy Bell      568  158    20   89  75    73    15   8068  2273    177
1045
## -Barry Bonds     413   92    16   72  48    65     1    413    92     16
72
## -Bobby Bonilla   426  109     3   55  43    62     1    426   109      3
55
##                 CRBI CWalks League Division PutOuts Assists Errors Salary
## -Alvin Davis     266    263      A        W     880      82     14    480
## -Andre Dawson    838    354      N        E     200      11      3    500
## -Andy VanSlyke   204    203      N        E     211      11      7    550
## -Buddy Bell      993    732      N        W     105     290     10    775
## -Barry Bonds      48     65      N        E     280       9      5    100
## -Bobby Bonilla    43     62      A        W     361      22      2    115
##                 NewLeague
## -Alvin Davis            A
## -Andre Dawson           N
## -Andy VanSlyke          N
## -Buddy Bell             N
## -Barry Bonds            N
## -Bobby Bonilla          N
```

3. Perform the best subset selection on the training sample. While choosing the best model fit up to a 19 variable model. Choose the best model according to adjusted R squared and Cp values. In case the best model differs according to the selection criteria, report the best fitted model according to each selection criterion.

```r
reg1 = regsubsets(Salary ~ ., data = train_data, nvmax = 19)
summary(reg1)
```

```
## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train_data, nvmax = 19)
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
```

```
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: exhaustive
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    " "   " "    "*"
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    " "
"*"
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    " "
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 7  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 8  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 9  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 10  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 11  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 12  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 13  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    " "   " "    "*"
"*"
## 14  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    " "   " "    "*"
"*"
## 15  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 16  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    "*"   "*"    "*"
```

```
"*"
## 17  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 5  ( 1 )  " "    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 7  ( 1 )  " "    " "     "*"       "*"     "*"     " "    " "
## 8  ( 1 )  " "    " "     "*"       "*"     "*"     "*"    " "
## 9  ( 1 )  " "    " "     "*"       "*"     "*"     "*"    "*"
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

```
reg1_sum = summary(reg1)

adjr2_best = which.max(reg1_sum$adjr2)
cp_best    = which.min(reg1_sum$cp)

adjr2_best

## [1] 12

cp_best

## [1] 11
```

Prediction Function

```
predict_regsubsets = function(object, newdata, id){
  form = as.formula(object$call[[2]])
  mat = model.matrix(form, newdata)
  coefi = coef(object, id = id)
  vars = names(coefi)
  mat[, vars] %*% coefi
}
```

MSE Function

```
mse = function(y, yhat){
  mean((y - yhat)^2)
}
```

4. Find the training MSE and test MSE for each of the best models selected according to different criterion as mentioned in 3 and compare their values.

```
# AdjR2 model
train_pred_adjr2 = predict_regsubsets(reg1, train_data, adjr2_best)
test_pred_adjr2  = predict_regsubsets(reg1, test_data, adjr2_best)

# Cp model
train_pred_cp = predict_regsubsets(reg1, train_data, cp_best)
test_pred_cp  = predict_regsubsets(reg1, test_data, cp_best)

data.frame(
  Model = c("AdjR2","Cp"),
  Train_MSE = c(
    mse(train_data$Salary, train_pred_adjr2),
    mse(train_data$Salary, train_pred_cp)
  ),
  Test_MSE = c(
    mse(test_data$Salary, test_pred_adjr2),
    mse(test_data$Salary, test_pred_cp)
  )
)

##   Model Train_MSE Test_MSE
## 1 AdjR2  88935.80 130066.9
## 2    Cp  89790.63 125652.3
```

5. Repeat 3-4 using forward stepwise selection method.

FORWARD STEPWISE SELECTION

```
reg2 = regsubsets(Salary ~ ., data = train_data, nvmax = 19, method =
'forward')
summary(reg2)

## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train_data, nvmax = 19,
##     method = "forward")
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
```

```
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: forward
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    " "   " "    "*"
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    " "   " "    "*"
"*"
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    " "   " "    "*"
"*"
## 7  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 8  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 9  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 10  ( 1 ) " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 11  ( 1 ) " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    "*"
"*"
## 12  ( 1 ) " "   " "  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 13  ( 1 ) "*"   " "  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 14  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   " "    "*"
"*"
## 15  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 16  ( 1 ) "*"   "*"  " "   "*"  " " "*"   "*"   "*"    "*"   "*"    "*"
"*"
```

```
## 17  ( 1 ) "*"   "*"  " "   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  "*"    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 5  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 7  ( 1 )  "*"    " "     "*"       " "     " "     " "    " "
## 8  ( 1 )  "*"    " "     "*"       "*"     " "     " "    " "
## 9  ( 1 )  "*"    " "     "*"       "*"     " "     " "    "*"
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

Selecting Best Model

```r
reg2_sum = summary(reg2)

adjr2_f = which.max(reg2_sum$adjr2)
cp_f    = which.min(reg2_sum$cp)

adjr2_f

## [1] 14

cp_f

## [1] 11
```

MSE for Forward

```r
train_pred_f = predict_regsubsets(reg2, train_data, adjr2_f)
test_pred_f  = predict_regsubsets(reg2, test_data, adjr2_f)

data.frame(
  Method = "Forward",
  Train_MSE = mse(train_data$Salary, train_pred_f),
```

```
  Test_MSE = mse(test_data$Salary, test_pred_f)
)

##    Method Train_MSE Test_MSE
## 1 Forward  88287.08 134001.7
```

6. Repeat 3-4 using backward stepwise selection method.

BACKWARD STEPWISE SELECTION

```
reg3 = regsubsets(Salary ~ ., data = train_data, nvmax = 19, method =
"backward")
summary(reg3)

## Subset selection object
## Call: regsubsets.formula(Salary ~ ., data = train_data, nvmax = 19,
##     method = "backward")
## 19 Variables  (and intercept)
##            Forced in Forced out
## AtBat          FALSE      FALSE
## Hits           FALSE      FALSE
## HmRun          FALSE      FALSE
## Runs           FALSE      FALSE
## RBI            FALSE      FALSE
## Walks          FALSE      FALSE
## Years          FALSE      FALSE
## CAtBat         FALSE      FALSE
## CHits          FALSE      FALSE
## CHmRun         FALSE      FALSE
## CRuns          FALSE      FALSE
## CRBI           FALSE      FALSE
## CWalks         FALSE      FALSE
## LeagueN        FALSE      FALSE
## DivisionW      FALSE      FALSE
## PutOuts        FALSE      FALSE
## Assists        FALSE      FALSE
## Errors         FALSE      FALSE
## NewLeagueN     FALSE      FALSE
## 1 subsets of each size up to 19
## Selection Algorithm: backward
##           AtBat Hits HmRun Runs RBI Walks Years CAtBat CHits CHmRun CRuns
CRBI
## 1  ( 1 )  " "   " "  " "   " "  " " " "   " "   " "    "*"   " "    " "
" "
## 2  ( 1 )  " "   " "  " "   " "  " " "*"   " "   " "    "*"   " "    " "
" "
## 3  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   " "    " "
" "
## 4  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 5  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
```

```
" "
## 6  ( 1 )  " "   " "  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 7  ( 1 )  " "   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 8  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 9  ( 1 )  "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 10  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    " "
" "
## 11  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 12  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 13  ( 1 ) "*"   "*"  " "   " "  " " "*"   " "   "*"    "*"   "*"    "*"
" "
## 14  ( 1 ) "*"   "*"  " "   " "  " " "*"   "*"   "*"    "*"   "*"    "*"
" "
## 15  ( 1 ) "*"   "*"  " "   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
" "
## 16  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
" "
## 17  ( 1 ) "*"   "*"  "*"   " "  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 18  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
## 19  ( 1 ) "*"   "*"  "*"   "*"  "*" "*"   "*"   "*"    "*"   "*"    "*"
"*"
##           CWalks LeagueN DivisionW PutOuts Assists Errors NewLeagueN
## 1  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 2  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 3  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 4  ( 1 )  " "    " "     " "       " "     " "     " "    " "
## 5  ( 1 )  " "    " "     "*"       " "     " "     " "    " "
## 6  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 7  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 8  ( 1 )  " "    " "     "*"       "*"     " "     " "    " "
## 9  ( 1 )  " "    " "     "*"       "*"     "*"     " "    " "
## 10  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 11  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    " "
## 12  ( 1 ) "*"    " "     "*"       "*"     "*"     " "    "*"
## 13  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 14  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 15  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 16  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 17  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 18  ( 1 ) "*"    " "     "*"       "*"     "*"     "*"    "*"
## 19  ( 1 ) "*"    "*"     "*"       "*"     "*"     "*"    "*"
```

Select Best Model

```
reg3_sum = summary(reg3)

adjr2_b = which.max(reg3_sum$adjr2)
cp_b    = which.min(reg3_sum$cp)

adjr2_b

## [1] 13

cp_b

## [1] 13
```

MSE for Backward

```
train_pred_b = predict_regsubsets(reg3, train_data, adjr2_b)
test_pred_b  = predict_regsubsets(reg3, test_data, adjr2_b)

data.frame(
  Method = "Backward",
  Train_MSE = mse(train_data$Salary, train_pred_b),
  Test_MSE = mse(test_data$Salary, test_pred_b)
)

##     Method Train_MSE Test_MSE
## 1 Backward  88865.02 134736.7
```

7. Compare the results of the three methods in terms of choice of best model, train MSE amd test MSE.

Comparison of Train MSE

Training MSE Comparison

Forward Selection: 88,287.08 (lowest)

Backward Selection: 88,865.02

Best Subset (Adj R^2): 88,935.80

Best Subset (Cp): 89,790.63 (highest)

Forward Selection has the lowest training MSE, which means it fits the training data slightly better than the other methods.

Comparison of Test MSE

Best Subset (Cp): 125,652.3 (lowest)

Best Subset (Adj R^2): 130,066.9

Forward Selection: 134,001.7

Backward Selection: 134,736.7 (highest)

The Best Subset model selected using the Cp criterion has the lowest test MSE, indicating the best performance on unseen data.

Interpretation

Although Forward Selection achieves the lowest training MSE, its higher test MSE suggests slight overfitting. Backward Selection performs similarly but is marginally worse.

Best Subset Selection, particularly using the Cp criterion, strikes a better balance between bias and variance, leading to improved generalization.

Final Conclusion

The Best Subset Selection model (Cp criterion) is the best overall model because it has the lowest test MSE.

This indicates that:

Exhaustive evaluation of all possible models (Best Subset) can lead to better generalization.

Stepwise methods (Forward and Backward Selection) are computationally efficient but may not always produce the optimal model.

##Problem Set 7: Shrinkage Methods

We would be working with the Hitters dataset available in the ISLR library. We have to predict the Salary based on all available variables. First, we make sure that we delete all rows with one or more missing values.

1. We split the data set into a training set and a test set randomly, keeping 100 observations in the test set and the remaining in the training set.

```
#install.packages("glmnet")

library(ISLR)
library(glmnet)

## Warning: package 'glmnet' was built under R version 4.5.2

## Loading required package: Matrix

## Warning: package 'Matrix' was built under R version 4.5.2

## Loaded glmnet 4.1-10

data(Hitters)
```

```r
# Remove missing values
Hitters = na.omit(Hitters)

dim(Hitters)

## [1] 263  20

set.seed(123)

test_index = sample(1:nrow(Hitters), 100)

test_data = Hitters[test_index, ]
train_data = Hitters[-test_index, ]

dim(train_data)

## [1] 163  20

dim(test_data)

## [1] 100  20

# Create model matrices (needed for glmnet)
x_train = model.matrix(Salary ~ ., train_data)[, -1]
y_train = train_data$Salary

x_test = model.matrix(Salary ~ ., test_data)[, -1]
y_test = test_data$Salary
```

2. We train the model using least squares and report the train MSE and test MSE.

```r
lm_model = lm(Salary ~ ., data = train_data)
summary(lm_model)

##
## Call:
## lm(formula = Salary ~ ., data = train_data)
##
## Residuals:
##     Min      1Q  Median      3Q     Max
## -809.76 -182.13  -54.69  138.36 1837.14
##
## Coefficients:
##               Estimate Std. Error t value Pr(>|t|)
## (Intercept)  212.67449  129.87001   1.638 0.103705
## AtBat         -2.04895    0.87954  -2.330 0.021229 *
## Hits           5.85495    3.15579   1.855 0.065612 .
## HmRun          5.43622    9.22155   0.590 0.556448
## Runs          -2.49915    4.03131  -0.620 0.536288
## RBI            2.32756    3.84573   0.605 0.545984
## Walks          6.40986    2.41415   2.655 0.008827 **
```

```
## Years         -8.30051   17.14327  -0.484 0.628996
## CAtBat         0.01569    0.19354   0.081 0.935512
## CHits         -0.54632    1.02039  -0.535 0.593204
## CHmRun        -2.38189    2.49628  -0.954 0.341604
## CRuns          1.56320    1.05024   1.488 0.138840
## CRBI           1.26993    1.02979   1.233 0.219524
## CWalks        -0.95946    0.45825  -2.094 0.038049 *
## LeagueN       44.32184   98.13766   0.452 0.652220
## DivisionW   -149.54532   56.87996  -2.629 0.009496 **
## PutOuts        0.41589    0.11170   3.723 0.000282 ***
## Assists        0.43589    0.28787   1.514 0.132176
## Errors        -5.50162    5.83866  -0.942 0.347642
## NewLeagueN   -15.79562   97.46148  -0.162 0.871479
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
##
## Residual standard error: 346.5 on 143 degrees of freedom
## Multiple R-squared:  0.4782, Adjusted R-squared:  0.4088
## F-statistic: 6.897 on 19 and 143 DF,  p-value: 1.144e-12
```

```r
# Predictions
# Predictions
train_pred_lm = predict(lm_model, newdata = train_data)
test_pred_lm  = predict(lm_model, newdata = test_data)

# MSE function
mse = function(y, yhat){
  mean((y - yhat)^2, na.rm = TRUE)
}

# Compute MSE
train_mse_lm = mse(train_data$Salary, train_pred_lm)
test_mse_lm  = mse(test_data$Salary, test_pred_lm)

train_mse_lm
```

```
## [1] 105334.2
```

```r
test_mse_lm
```

```
## [1] 87803.91
```

3.We fit ridge regression models on the training set, with grid values of λ. We report the following:

(i) the dimension of the coefficient matrix so obtained.

(ii) The l2 norm of the regression coefficients corresponding to the 50th and 60th choice of λ. We comment on the output.

(iii) Test MSE corresponding to these two choices of λ.

```
lambda_grid = 10^seq(10, -2, length = 100)

ridge_model = glmnet(x_train, y_train, alpha = 0, lambda = lambda_grid)
```

(i) Dimension of Coefficient Matrix

```
dim(coef(ridge_model))

## [1]  20 100
```

Coefficients for 50th and 60th λ

```
coef_50 = coef(ridge_model)[, 50]
coef_60 = coef(ridge_model)[, 60]

coef_50

##   (Intercept)         AtBat          Hits         HmRun          Runs
## 438.509969440   0.029684297   0.110523230   0.560867468   0.178502584
##           RBI         Walks         Years        CAtBat         CHits
##   0.252163513   0.238337160   1.002466622   0.002796464   0.010194612
##        CHmRun         CRuns          CRBI        CWalks       LeagueN
##   0.077846378   0.020089803   0.021957346   0.021866228   0.432252998
##     DivisionW       PutOuts       Assists        Errors    NewLeagueN
##  -6.916280599   0.018294166   0.002998934  -0.082552703   0.741567242

coef_60

##  (Intercept)        AtBat         Hits        HmRun         Runs
RBI
## 105.63996577   0.08310078   0.51246876   2.09635144   0.60993278
1.14623958
##        Walks        Years       CAtBat        CHits       CHmRun
CRuns
##   1.01677160   2.77366963   0.01043635   0.04276159   0.27032634
0.08012067
##         CRBI       CWalks      LeagueN    DivisionW      PutOuts
Assists
##   0.09127173   0.05840151  18.35350369 -62.46410535   0.14784491
0.04002532
##       Errors   NewLeagueN
##  -0.86723778  12.21693330
```

(ii). L2 Norm of Coefficients

```
l2_norm = function(beta){
  sqrt(sum(beta^2))
}

l2_50 = l2_norm(coef_50)
```

```r
l2_60 = l2_norm(coef_60)

l2_50

## [1] 438.5671

l2_60

## [1] 124.7542
```

(iii). Test MSE for 50th and 60th λ

```r
# Predictions
ridge_pred_50 = predict(ridge_model, s = lambda_grid[50], newx = x_test)
ridge_pred_60 = predict(ridge_model, s = lambda_grid[60], newx = x_test)

# Test MSE
mse_50 = mse(y_test, ridge_pred_50)
mse_60 = mse(y_test, ridge_pred_60)

mse_50

## [1] 167303.1

mse_60

## [1] 97935.56
```

Least Squares: Fits the data without any regularization (no shrinkage).

Can lead to overfitting, especially when the model is complex.

Typically reports both training MSE and test MSE for evaluation.

Ridge Regression:

Applies a penalty term (λ) to shrink coefficients toward zero.

As λ increases, coefficients become smaller.

This introduces higher bias but reduces variance, improving model stability.

Results Interpretation

The L2 norm decreases as λ increases, meaning the overall magnitude of coefficients reduces.

For example, the model with the 50th λ has larger coefficients compared to the 60th λ.

Test MSE is used to identify the optimal value of λ that provides the best generalization performance.

Conclusion:

Ridge regression improves model stability by reducing variance.

The optimal λ is the one that minimizes the test MSE.

Compared to Least Squares, Ridge Regression often performs better on unseen (test) data due to its regularization effect.